

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SDMCET, DHARWAD**

**MINOR PROJECT – I
SYNOPSIS**

**DESIGN AND DEVELOPMENT OF RESUME ANALYZER AND
REVIEWER**

Course Name: Minor-Project-I

Course Code: 22UCSL505

Semester: V

Academic Year: 2026–27

Team Size: 04

Front-end: React JS, Tailwind CSS

Back-end: FastAPI (Python)

Database: PostgreSQL

Machine Learning: scikit-learn

Anirudh S : 2SD24CS010

Chinmayee S : 2SD24CS017

Gururajagouda : 2SD24CS021

Suhas S M : 2SD24CS102

Submitted in partial fulfillment of the requirements for Minor Project–I

ABSTRACT

Resume screening is an important stage of recruitment and placement, but manually comparing resumes with job descriptions is time-consuming and may produce inconsistent results. The proposed Resume Analyzer and Developer is a web-based system designed to help students and job seekers evaluate and improve their resumes for a target role. The system accepts resume information or document text along with a job description, preprocesses the text, identifies important sections and skills, and compares candidate information with job requirements. Natural Language Processing and machine-learning techniques, particularly TF-IDF and cosine similarity with explicit skill matching, will be used to generate an interpretable relevance score. The system will highlight matched skills, missing requirements, strengths and areas for improvement. React JS and Tailwind CSS will provide the user interface, FastAPI and Python will implement backend services, PostgreSQL will store relevant data, and scikit-learn will support analysis. The expected outcome is a practical placement-preparation tool that enables iterative resume improvement and job-role alignment.

1. INTRODUCTION

A resume is a semi-structured document that presents a candidate's education, technical skills, projects, experience, certifications and achievements. In modern recruitment and campus placement, the ability to communicate these details clearly is important because recruiters and automated screening systems often evaluate a large number of applications. Resumes can differ substantially in formatting, section names, ordering and terminology, which makes automatic processing a challenging document-analysis problem. A useful system must first identify relevant information from the resume and then determine how well that information corresponds to the requirements of a target job. This is particularly useful for students and fresh graduates, who may have limited professional experience but possess academic projects, technical skills and certifications that need to be presented effectively. The proposed project therefore treats the resume not only as a document to be screened, but also as a profile that can be analyzed and improved. By providing understandable feedback rather than only a numerical score, the system aims to support students during placement preparation and repeated resume revision.

Automated resume processing has been studied using information extraction, machine learning and Natural Language Processing. Ayishathahira, Sreejith and Raseek investigated a combination of neural networks and Conditional Random Fields for efficient resume parsing [1]. Their work shows the importance of extracting structured information from resume text before meaningful matching can be performed. Maheshwary and Misra studied resume-to-job matching using a deep Siamese network, where resumes and job descriptions are represented so that semantically similar pairs can be brought closer in a learned representation space [2]. This is relevant because simple keyword matching may fail when a resume and a job description use different expressions for related skills or competencies. Li et al. investigated context-aware transformer models for competence-level prediction and resume–job-description matching [3]. Such approaches demonstrate the potential of advanced semantic models for recruitment-oriented applications, while also highlighting the need for suitable datasets, evaluation and careful interpretation. These studies establish the state of the art from resume parsing to semantic matching and provide a basis for selecting an appropriate baseline for the proposed minor project.

1.1 Literature Survey

- Resume parsing research demonstrates the need to convert semi-structured resume text into useful entities and sections before matching [1].
- Deep-learning approaches to resume–job matching indicate that semantic similarity can improve upon exact word overlap [2].
- Context-aware transformer models demonstrate the potential of advanced NLP for competence prediction and job matching [3].
- Information-retrieval methods such as TF-IDF and vector-space similarity provide an interpretable and practical baseline for text comparison [4].

1. INTRODUCTION (CONTINUED)

Despite these developments, many recruitment-oriented systems concentrate mainly on screening, ranking or predicting suitability. Students, however, require an additional step: they need to understand why a resume is not well aligned with a target role and what can be improved. The research gap addressed by this project is therefore the integration of resume analysis with a student-oriented resume development workflow. The proposed system will compare the resume with a specified job description, identify important skills and requirements, calculate an interpretable relevance score and present the result in an actionable form. Instead of treating the score as a final decision, the application will use it as guidance for iterative improvement. A student can revise the resume, run the analysis again and observe whether the alignment has improved. This approach combines document processing, information extraction, text similarity and web-based application development into a single workflow. The initial implementation will intentionally use a manageable and explainable baseline so that the project can be implemented, tested and evaluated within the scope of Minor Project–I.

The proposed work follows a modular architecture consisting of a presentation layer, application layer, analysis layer and data layer. The presentation layer will be implemented using React JS and Tailwind CSS to provide forms, resume input, job-description input and result dashboards. FastAPI with Python will provide backend APIs for validation, processing and communication between the interface and analytical components. The analysis module will perform text preprocessing, feature representation and matching using Python and scikit-learn. PostgreSQL will maintain user, resume, job-description and analysis records required for repeated evaluation. The functional workflow begins with user input and validation, followed by preprocessing and information extraction. The system then performs skill and document-level matching, generates a score and gap analysis, prepares recommendations and stores the required results. The modular design also makes it possible to replace or improve the baseline NLP component in future work without redesigning the complete application.

1.2 Proposed Work

The proposed project will develop a web-based Resume Analyzer and Developer that accepts resume content and a target job description, analyzes their alignment and provides improvement-oriented feedback. The first version will focus on major resume sections, technical skills, job requirements, TF-IDF-based representation, cosine similarity, explicit skill matching and a basic recommendation engine. The system will be designed for students and fresh graduates preparing for placements and entry-level opportunities.

1.3 Objectives

- Develop a web-based platform for resume analysis and improvement.
- Process major resume sections such as education, skills, projects, experience and certifications.
- Analyze a target job description and identify important requirements and skills.
- Compare resume and job-description text using text-similarity and machine-learning techniques.
- Generate an interpretable relevance score with matched and missing requirements.
- Provide practical suggestions for improving resume content and job-role alignment.

- Integrate React JS, FastAPI, PostgreSQL and scikit-learn into one working application.

2. METHODOLOGY

The project will be developed incrementally so that each component can be implemented and verified before complete integration. Requirement analysis and literature study will be followed by interface design, database design, text preprocessing, matching implementation, backend development, frontend integration and testing. A simple baseline will be established first and later improvements, if any, will be measured against it. This approach reduces implementation risk and keeps the analytical process understandable.

2.1 System Design and Modules

- User/Profile Module – manages basic profile information and the target job role.
- Resume Module – accepts resume content and organizes major sections such as education, skills, projects and experience.
- Job Description Module – accepts and prepares job requirements for analysis.
- Preprocessing Module – cleans and normalizes resume and job-description text before comparison.
- Information Extraction Module – identifies relevant skills and important resume sections.
- Matching Module – calculates document similarity and skill alignment.
- Feedback Module – converts analysis findings into understandable resume improvement suggestions.
- Database Module – stores selected profile, resume, job and analysis information in PostgreSQL.
- Frontend Module – presents input forms, analysis results, matched skills, gaps and recommendations.

2.2 Text Processing

Resume and job-description text will be converted into a common representation using cleaning and normalization operations. Depending on the input format, preprocessing may include text extraction, case normalization, whitespace cleanup, punctuation handling and tokenization. Terms with domain meaning, such as programming languages, frameworks, databases and certifications, should be retained. The same processing strategy will be applied consistently to both document types so that the comparison remains meaningful.

2.3 Feature Representation and Matching

Term Frequency–Inverse Document Frequency (TF-IDF) will be used as the practical baseline for representing documents numerically. TF-IDF gives greater importance to informative terms while reducing the influence of terms that occur very frequently across documents. After vectorization, cosine similarity will estimate the relationship between the resume and job-description vectors.

$$\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

The project will also include explicit skill matching. The system can count required skills found in the resume and use this information together with document-level similarity. Exact weights

and score thresholds will be selected only after experimentation and testing; no performance percentage will be claimed before experimental evaluation.

2.4 Database Design

Entity	Representative Fields	Purpose
User	user_id, name, email, target_role	User identification and target-role information
Resume	resume_id, user_id, content, version	Resume storage and revision
Job	job_id, role, description	Target job data
Analysis	analysis_id, resume_id, job_id, score	Analysis result
Skill Match	analysis_id, skill, status	Matched and missing skill details

2. METHODOLOGY (CONTINUED)

2.5 API and Frontend Integration

FastAPI will expose backend endpoints for submitting resumes, submitting job descriptions, requesting analysis and retrieving results. Validation will occur at the API boundary to ensure that incomplete or invalid information is handled predictably. React JS components will communicate with these APIs and display the returned structured data. Tailwind CSS will be used for a consistent and responsive interface. The application will provide clear status messages for upload errors, missing information and successful analysis.

2.6 Functional Flow

User input → validation → resume/job preprocessing → feature extraction → skill and similarity matching → score and gap analysis → recommendation generation → storage → dashboard presentation. The user can revise the resume and repeat the analysis, creating an iterative improvement cycle.

2.7 Software and Hardware Requirements

- Hardware: A standard development laptop or desktop with sufficient RAM and storage for the IDE, backend, database and browser.
- Operating System: Windows, Linux or macOS.
- Frontend: React JS and Tailwind CSS.
- Backend: Python with FastAPI.
- Database: PostgreSQL.
- Machine Learning/NLP: Python and scikit-learn.
- Development Environment: Visual Studio Code or an equivalent IDE.
- Version Control: Git/GitHub may be used for version control and team collaboration.

2.8 Development Phases

- Requirement analysis and literature study.
- UI wireframe and database design.

- Resume/job preprocessing and baseline matching.
- FastAPI and PostgreSQL implementation.
- React frontend integration and dashboard development.
- Testing, evaluation, refinement and documentation.

2.9 Testing and Evaluation

Unit testing will verify individual preprocessing, extraction and scoring functions. Integration testing will check data flow among React, FastAPI, the analysis module and PostgreSQL. System testing will cover complete user scenarios, including valid resumes, incomplete resumes, short job descriptions, unrelated skill sets, missing required fields and repeated analysis of revised resumes. If a labelled dataset is used, accuracy, precision, recall and F1-score may be calculated. If the first implementation is primarily similarity-based, manually prepared resume–job pairs can be used to examine whether stronger skill alignment generally produces higher scores. User evaluation can assess clarity, relevance and usefulness of the feedback.

3. OUTCOMES

3.1 Student-Specific Outcomes (Learning Outcomes)

- Gain practical experience in full-stack web application development.
- Apply NLP and machine-learning concepts to a real document-analysis problem.
- Understand API design, database integration and frontend–backend communication.
- Develop skills in testing, debugging, modular design and technical documentation.
- Understand how literature-survey findings can guide a practical software implementation.
- Gain experience in integrating an analytical component with a database-backed web application.

3.2 Project-Specific Outcomes

- A working web application for analyzing a resume against a target job description.
- Processing of major resume sections and relevant technical skills.
- An interpretable baseline relevance score based on text similarity and skill alignment.
- Identification of matched skills and important skill gaps.
- Resume improvement recommendations based on the analysis findings.
- Persistent storage of relevant resume, job and analysis data.
- An iterative workflow that allows users to revise and re-analyze their resumes.

3.3 Applications

The system can support students and fresh graduates during placement preparation, job applications and resume revision. Educational institutions can adapt the workflow as a training tool for placement cells. The architecture can also be extended to compare one profile with multiple jobs, recommend suitable roles and track resume versions over time.

3.4 Limitations and Future Scope

The quality of results will depend on input quality, document structure and available evaluation data. A TF-IDF baseline may not capture all semantic relationships, synonyms or implied competencies, and resume parsing may be affected by unusual formatting. Therefore, the score should be treated as a guidance measure rather than a definitive judgment of candidate quality. Future versions can investigate semantic embeddings, transformer-based matching, multilingual resume support, explainable scoring, multiple-job comparison, role recommendation, skill-gap learning paths and resume-version comparison.

3.5 Conclusion

The proposed Resume Analyzer and Developer addresses a practical problem faced by students and job seekers: understanding how well a resume aligns with a target job and deciding what to improve. The project combines document processing, text similarity, database management and full-stack web development in one system. Its layered design separates the presentation, application, analysis and data components, making the system easier to implement, test and extend. The initial TF-IDF, cosine similarity and explicit skill-matching baseline is intentionally interpretable and appropriate for a minor project. The completed system is expected to provide a useful placement-preparation tool while giving the project team practical experience in software engineering, NLP, machine learning and full-stack development.

4. REFERENCES

- [1] C. H. Ayishathahira, C. Sreejith, and C. Raseek, “Combination of Neural Networks and Conditional Random Fields for Efficient Resume Parsing,” in Proc. 2018 International CET Conference on Control, Communication, and Computing (IC4), 2018, pp. 388–393, doi: 10.1109/CETIC4.2018.8530883.
- [2] S. Maheshwary and H. Misra, “Matching Resumes to Jobs via Deep Siamese Network,” in Proc. 2018 World Wide Web Conference (WWW), Lyon, France, 2018, doi: 10.1145/3184558.3186942.
- [3] C. Li, E. Fisher, R. Thomas, S. Pittard, V. Hertzberg, and J. D. Choi, “Competence-Level Prediction and Resume & Job Description Matching Using Context-Aware Transformer Models,” in Proc. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 8456–8466, doi: 10.18653/v1/2020.emnlp-main.679.
- [4] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.
- [5] T. Joachims, “Text categorization with Support Vector Machines: Learning with many relevant features,” in Machine Learning: ECML-98, Lecture Notes in Computer Science, vol. 1398, 1998, pp. 137–142.